

KISAN.AI Backend — Sequential Flow & Integration Plan (FastAPI + Node.js)

This blueprint gives you a **step-by-step**, production-ready flow using: scraping of government portals, Plant.health API, SerpAPI, local agri portals, government schemes portals, Agmarknet (data.gov.in) API, Google STT/TTS for voice + interruption handling, OpenWeather API, Location/Maps API, and room for other APIs.

0) High-Level Components

- **Client (Web/Android):** Voice + camera + location permissions.
 - **Node.js Gateway** (Express): Auth, rate limiting, session, STT/TTS streaming proxy, Socket.io streaming to client.
 - **Python (FastAPI) Core:** Agent router + feature micro-routers (Disease, Market, Schemes, Remedies, Weather).
 - **Agents/Services** (Python):
 - `plant_health` : Plant.health image diagnosis (+ optional fallback Google Vision/Vertex AI).
 - `price_compare` : SerpAPI + scrapers (Apify/Playwright) + marketplace/ecommerce APIs.
 - `agmarknet` : data.gov.in APIs for mandi prices; optional eNAM/state portals.
 - `schemes_rag` : RAG over central/state portals (Docling → embeddings → vector DB/Neo4j).
 - `weather` : OpenWeather/IMD; risk scoring + irrigation reminders.
 - `location` : Google Maps Geocode/Places; nearest mandi/state mapping.
 - **Data Stores:** MongoDB (user, sessions, cached results), Neo4j (graph RAG), Vector DB (Chroma/Weaviate), Redis (cache, queues), Object Storage (images, transcripts).
 - **Workers:** Celery/RQ for async scraping, enrichment, retrievals.
 - **Observability:** Structured logs, metrics, traces, alerting.
-

1) End-to-End Session (Voice + Image) — Sequential Flow

1. **Start Session (Client)**
2. User taps mic or camera.
3. Client captures GPS (coarse), language preference.
4. Socket.io connects to Node.js; JWT attached.
5. **STT Streaming (Node.js)**
6. Audio chunks → Google STT (streaming) via gateway.
7. Partial transcripts streamed back to client (Socket.io) for UI captions.
8. **Interruption handling:** If user speaks while TTS is playing, client sends `interrupt` → Node cancels current TTS stream and notifies FastAPI to pause agent.

9. Intent Routing (FastAPI)

10. Text (from STT) + context (image? GPS? crop?) → `router/main`.

11. Lightweight classifier (rule+LLM) picks **agent**: `disease | market | schemes | remedies | weather`.

12. Agent Execution (FastAPI → Services)

13. Agent calls external APIs/scrapers, normalizes data, computes confidence, attaches sources.

14. Long tasks offloaded to Celery (e.g., scraping multiple portals) with job IDs; streaming progress via Socket.io.

15. Response Orchestration

16. Unified answer (short farmer-friendly summary + details + sources + confidence score).

17. Localization: translate to user language; numbers in local units.

18. TTS Playback (Node.js)

19. Summary text → Google TTS (streaming); client plays audio and shows formatted cards.

20. Feedback + RL Loop

21. Thumbs up/down; optional corrective text/voice → stored for continuous improvement.

2) Feature Flow A — Disease Diagnosis → Cheapest Remedy

Inputs: Image (leaf/plant), crop (optional), location.

Step-by-Step: 1. **Validate & Store Image** (Node → GCS/Cloud Storage) → signed URL. 2. **Detect Disease** (FastAPI → `services/plant_health.detect_disease(url)`) - Primary: Plant.health API (returns disease name, score, alternative hypotheses). - Fallback (optional): Google Vision labels + your classifier (MobileNet/EfficientNet on PlantVillage) → disease mapping. 3. **Map Disease → Active Ingredients** - Lookup table (YAML/DB): disease → {AI list, recommended formulations, dosage ranges} + organic alternatives. 4. **Price Discovery (Cheapest Remedy)** - **SerpAPI** queries (Google) with structured operators (site filters, commodity keywords, location suffix): - e.g., `"Mancozeb 75% WP price" site:bighaat.com OR site:agrostar.in AND "500 g"` - **Scraping layer** (Celery workers): - Playwright/Apify actors for IndiaMART, state agri cooperatives, local portals. - **Normalization:** Convert price to ₹/kg or ₹/L; parse pack sizes; handle ranges. 5. **Subsidy/Application Check** - Query state/central portals datasets to see if the AI/product is subsidized. 6. **Rank & Recommend** - Score by **effective cost** (price – subsidy), availability, distance (shipping or local vendor), and environmental score. 7. **Output** - "Likely disease: *Late blight* (92%). Use **Mancozeb 75% WP** at 2 g/L. Cheapest nearby:

₹245/kg (Vendor X, Link). Organic option: Neem oil 3 ml/L (₹180/L). Sources: ..."

- Include QR/open-in-maps for vendor, and links.

Resilience: - Cache price results (Redis TTL 24–72h). - Fallback to last-known prices if live calls fail; flag staleness.

3) Feature Flow B — Real-Time Market Insights (Agmarknet + Trends)

Inputs: Crop, location (state/district/nearest mandis), timeframe.

Step-by-Step: 1. **Geocode** GPS → state/district; find nearest 3 mandis (static table + Maps API). 2. **Fetch Prices** via **data.gov.in Agmarknet** resource IDs (free API key): modal/min/max price, arrivals, date. 3. **Optionally** scrape **eNAM/state portals** for missing mandis. 4. **Store** in MongoDB time-series (per crop/mandi) for trend modeling. 5. **Analytics** - Compute 7/14/30-day moving averages, volatility, z-scores. - Basic seasonality heuristic by month; outlier removal. 6. **Recommendation Engine** - Rules/ML: If current price < MA7 but rising and forecast up next week → "hold"; else "sell". - Suggest top 2 mandis within 100 km with better modal price; show transport delta. 7. **Output** - Card: current price, trend arrows, forecast window, best mandi suggestion, confidence, sources (Agmarknet/eNAM).

Resilience: Cache per crop/mandi/day; backoff/retry on 429; keep last 90 days locally.

4) Feature Flow C — Govt Schemes Guidance (RAG over Central + State)

Inputs: Natural language question + state/district + profile (small/marginal farmer, landholding).

Step-by-Step: 1. **Crawl/Fetch** official portals (pmkisan, pmfby, agricoop, agrimachinery, dbtbharat, state dept portals). Use **Docling** to parse PDFs/HTML and extract clean text + metadata. 2. **Index** into Vector DB (embeddings) + **Neo4j** graph (entities: scheme → eligibility → docs → apply link → deadlines → state variants). 3. **Query:** Build retriever (BM25 + dense + graph hops) → gather top K chunks + node context. 4. **Synthesize:** LLM builds step-wise eligibility, required docs, benefits, and **direct application links**. 5. **Localize:** Translate to user language; simplify reading level. 6. **Output:** Show card with: "Eligible: Yes/No", reasons, steps, documents checklist, link buttons; sources with page anchors.

Freshness: Weekly crawl jobs; change detection; re-embed deltas; versioned sources.

5) Feature Flow D — Weather & Irrigation Alerts

1. **OpenWeather API** (or IMD): current + hourly/daily forecast by GPS.
2. **Crop-stage model:** growth stage (from user logs/images) → water need curve.

3. **Advisory:** If rainfall > threshold next 24h → delay irrigation; temp + humidity → disease risk.
4. **Notifications:** SMS/WhatsApp/Push + voice TTS; quiet hours honored.

6) Node.js Gateway Responsibilities

- **Auth:** JWT (short-lived) + refresh.
- **Rate Limiting:** IP + user + feature bucketed.
- **STT/TTS Streaming:** Proxy to Google **Speech-to-Text** and **Text-to-Speech**; handle barge-in:
 - When `interrupt` event arrives, abort current TTS stream; send `pause` to FastAPI agent; buffer user speech.
- **Uploads:** Image to Cloud Storage; return signed URLs.
- **WebSockets:** Progress events, partial answers, token streaming.

7) FastAPI Layout (Modules & Contracts)

Routers - `routers/main.py` — intent routing. - `routers/disease.py` — `/disease/diagnose` (POST image), `/disease/remedies` (GET by disease, location). - `routers/market.py` — `/market/prices` (GET crop,state,district), `/market/trends`. - `routers/schemes.py` — `/schemes/eligibility`, `/schemes/search`. - `routers/weather.py` — `/weather/forecast`.

Services - `services/plant_health.py`, `agmarknet.py`, `price_compare.py`, `schemes_rag.py`, `weather.py`, `location.py`.

Models (Pydantic) - `DiseaseRequest{ image_url, crop, location }` - `DiseaseResponse{ disease, confidence, alt, advice[], sources[] }` - `RemedyOption{ name, ai, dose, pack_size, price, price_per_unit, vendor, url, distance_km, subsidy_applicable }` - `MarketPrice{ mandi, date, min, max, modal, arrivals }` - `SchemeAnswer{ eligible: bool, summary, steps[], documents[], links[], sources[] }`

8) External API Integration Details

- **Plant.health:** REST; send image URL or multipart; store response; handle 4xx/5xx with retries.
- **SerpAPI:** Google engine; use `site:` filters; paginate; respect monthly quota; cache queries (hash of (query, location)).
- **Scraping:** Playwright (headless Chrome), rotating proxies; robots.txt aware; backoff; HTML → price parser (regex + heuristics) → normalized units.
- **Agmarknet (data.gov.in):** Register API key; resource IDs for mandi prices; schedule nightly backfills.
- **OpenWeather:** One Call API 3.0; per-GPS cache 30–60 min.
- **Location API:** Google Maps Geocoding & Places for nearest mandi; pre-seed a static mandi list.
- **Google STT/TTS:** Streaming gRPC; chunked audio; Hindi + regional language models; latency targets < 400ms for partials.

9) Caching, Cost, and Rate-Limit Strategy

- **Redis** tiers:
 - **R1** (short TTL: minutes): STT partials, weather.
 - **R2** (medium TTL: hours): price comparison results, SerpAPI results.
 - **R3** (long TTL: days): scheme docs metadata, mandi static lists.
 - **Budget guards**: Per-user/day caps on SerpAPI & Plant.health calls; graceful degradation to cached answers.
 - **Batching**: Combine queries (multi-mandi) per request to minimize API calls.
-

10) Confidence & Sources

- **Confidence scoring** = weighted mean of:
 - Model/API confidence (disease score).
 - Evidence density (no. of agreeing sources for price/scheme).
 - Freshness (days since updated).
 - Always show sources (domain + title); tag freshness.
-

11) Security & Compliance

- Secrets → GCP Secret Manager.
 - PII minimization; encrypt at rest (MongoDB) and in transit (TLS).
 - Respect ToS for scraping; honor robots.txt; throttle.
 - Clear consent for microphone, location, and image capture.
-

12) Observability

- **Logs**: JSON logs with `request_id`, `user_id`, `feature`, `latency_ms`, `api_cost`.
 - **Metrics**: p95 latency per feature; API error rates; cache hit ratio.
 - **Tracing**: OpenTelemetry across Node ↔ FastAPI ↔ workers.
-

13) Deployment (GCP-friendly)

- FastAPI + workers on **Cloud Run**; Redis via **MemoryStore**; MongoDB Atlas; Neo4j Aura; Cloud Storage for images.
 - CI/CD with Cloud Build; IaC via Terraform.
-

14) Sample Sequence Diagrams (textual)

Disease → Remedy 1) Client → Node: upload image, GPS → signed URL 2) Node → FastAPI `/disease/diagnose` 3) FastAPI → Plant.health 4) FastAPI → `price_compare` (SerpAPI + scrapers via Celery) 5) Workers → Mongo cache 6) FastAPI → Node: unified response (summary + cheapest options) 7) Node → Client: speak via TTS + cards

Market Insight 1) Client → FastAPI `/market/prices?crop=Onion&state=MH&district=Nashik` 2) FastAPI → Agmarknet + cache 3) FastAPI → trends → response with recommendations

Schemes (RAG) 1) Client → FastAPI `/schemes/eligibility?query=KCC&state=UP` 2) FastAPI → retriever (vector DB + Neo4j) 3) FastAPI → LLM synthesize → response with links

15) First Implementation Sprint (Checklist)

- [] Node gateway skeleton: auth, uploads, Socket.io, STT/TTS proxy
 - [] FastAPI skeleton + routers + health checks
 - [] Plant.health adapter + image flow E2E
 - [] Agmarknet adapter + price/trend endpoints
 - [] SerpAPI adapter + basic price compare + normalization
 - [] RAG bootstrap: crawl pmkisan/pmfby + Docling → embeddings → `/schemes/eligibility`
 - [] Redis caching + keys/TTLs
 - [] Observability: logs/metrics + cost counters
 - [] Confidence + sources formatter
 - [] Hindi+English TTS/STT configs + interruption handling
-

16) Minimal Pydantic Schemas (examples)

```
class DiseaseResponse(BaseModel):
    disease: str
    confidence: float
    alternatives: list[str] = []
    advice: list[str] = []
    sources: list[HttpUrl]

class RemedyOption(BaseModel):
    name: str
    active_ingredient: str
    dose: str # e.g., "2 g/L"
    pack_size: str # e.g., "500 g"
    price_inr: float
    price_per_unit_inr: float # normalized
    vendor: str
```

```

url: HttpUrl
distance_km: float | None
subsidy_applicable: bool

class MarketPrice(BaseModel):
    mandi: str
    state: str
    district: str
    date: date
    min: int
    modal: int
    max: int
    arrivals_qtl: float | None

class SchemeAnswer(BaseModel):
    eligible: bool
    summary: str
    steps: list[str]
    documents: list[str]
    links: list[HttpUrl]
    sources: list[HttpUrl]

```

17) Error Handling & Fallbacks

- Plant.health down → switch to Vision+local model; show lower confidence + caveats.
- SerpAPI quota hit → use cached prices; mark as (stale).
- Agmarknet 429/5xx → exponential backoff; show last 3 days median.
- STT network loss → continue with queued text input; resume when connection restores.

18) Localization & UX Notes

- Keep messages at a Class 6–8 reading level.
- Use short bullets in TTS; avoid long sentences.
- Units: default to local (₹/quintal, g/L); explain conversions when needed.

This plan is structured to let you **ship a working MVP quickly** and scale safely. Update any module names to match your repo layout, and you're good to go.